

[Home](#)[Newsletter](#)[Webinars](#)

How the Browser Became the Main Cyber Battleground

The Hacker News Jul 29, 2025

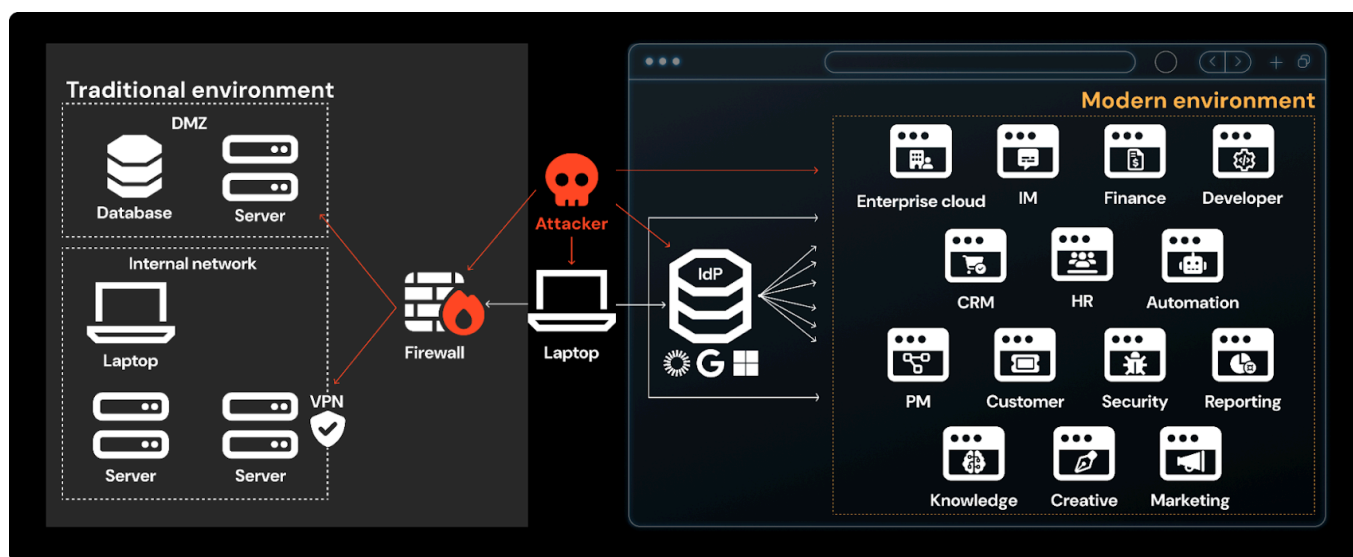


Until recently, the cyber attacker methodology behind the biggest breaches of the last decade or so has been pretty consistent:

- Compromise an endpoint via software exploit, or social engineering a user to run malware on their device;
- Find ways to move laterally inside the network and compromise privileged identities;
- Repeat as needed until you can execute your desired attack — usually stealing data from file shares, deploying ransomware, or both.

But attacks have fundamentally changed as networks have evolved. With the SaaS-ification of enterprise IT, core business systems aren't locally deployed and centrally managed in the way they used

to be. Instead, they're logged into over the internet, and accessed via a web browser.



— Attacks have shifted from targeting local networks to SaaS services, accessed through employee web browsers.

Under the shared responsibility model, the part that's left to the business consuming a SaaS service is mostly constrained to how they manage identities — the vehicle by which the app is accessed and used by the workforce. It's no surprise that this has become the soft underbelly in the crosshairs of attackers.

We've seen this time and again in the biggest breaches of recent years, with the highlights including the massive [Snowflake campaign in 2024](#) and the [2025 crime wave attributed to Scattered Spider](#).

These attacks are so successful because while attackers have moved with the changes to enterprise IT, security hasn't really kept up.

The browser is the new battleground — and a security blind spot

Taking over workforce identities is the first objective for attackers looking to target an organization, and the browser is the place where the attacks against users happen. This is because it's where these digital identities are created and used — and their credentials and sessions live. This is what the attacker wants to get their hands on.

Stolen credentials can be used as part of targeted attacks or in broader credential stuffing (cycling known username and credential pairs against various apps and platforms), while stolen session tokens can be used to log in directly to an active session, bypassing the authentication process.

There are a few different techniques that attackers can use to get access to these identities. Attackers harvest stolen credentials from various places — **data breach dumps, mass credential phishing campaigns, infostealer logs**, even **malicious browser extensions** that they've tricked an employee into installing. In fact, the cyber crime ecosystem itself has shifted on its axis to cater to this, with hackers specifically taking on the role of harvesting credentials and establishing account access for others to exploit.

The high-profile **Snowflake** breaches in 2024 signalled a watershed moment in the shift to identity-driven breaches, where attackers logged into accounts across hundreds of customer tenants using stolen credentials. One of the primary sources of the stolen credentials used in the attacks were infostealer logs dating back to 2020 — breached passwords that hadn't been rotated or mitigated with MFA.

Infostealers are notable because they're an endpoint malware attack designed to harvest credentials and session tokens (primarily from the browser) to enable the attacker to then log into those services... through their own web browser. **So, even today's endpoint attacks are seeing the attacker pivot back into the browser in order to get to identities — the key to the online apps and services where exploitable data and functionality now resides.**

Attacks in the browser vs. on the browser

There's an important distinction to be made between attacks that happen in the browser, vs. those happening against the browser itself.

There's growing consensus that **the browser is the new endpoint**. But the analogy isn't perfect — the reality is that web browsers have a comparatively limited attack surface compared to the complexity of the traditional endpoint — comparing something like Google Chrome with a Windows OS seems a very unbelievable concept.

Attacks that target the browser itself as a mechanism to compromise identities are few and far between. One of the more obvious vectors is using malicious browser extensions — so, scenarios in which a user has either:

- Been lured into installing an already malicious extension, or
- Is using a browser extension that is later compromised by an attacker

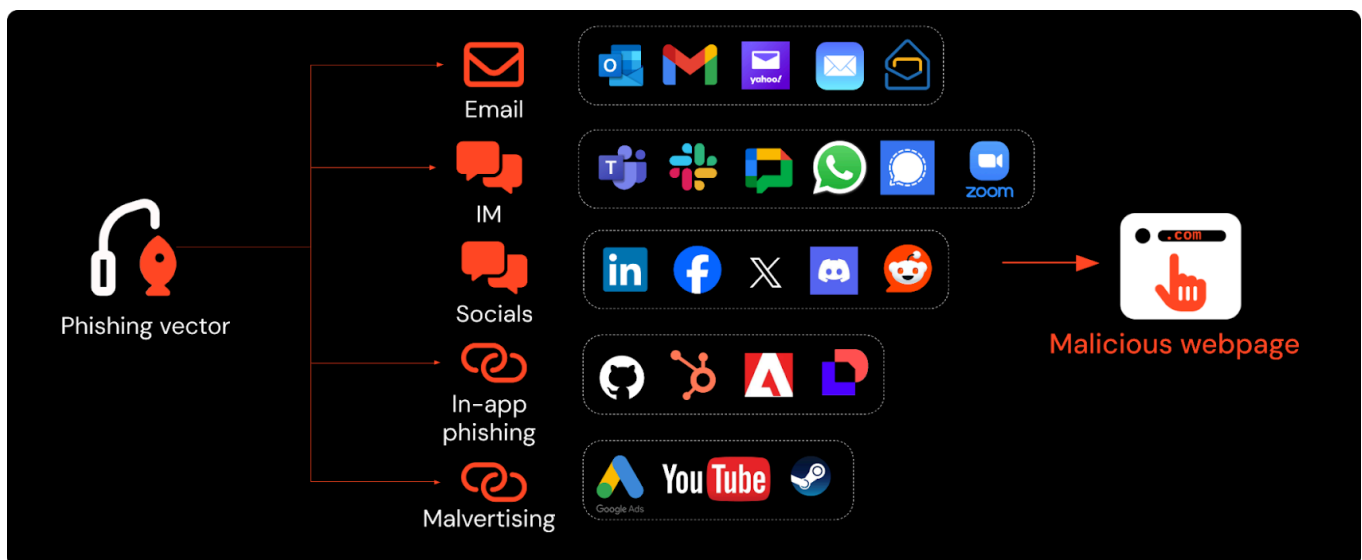
But the problem of malicious extensions is something you solve once, and then move on. The reality is that users should not be installing random browser extensions, and given the risk, you should:

- Lock down your environment to allow only a handful of essential extensions.
- Monitor for indicators that an extension you trust is compromised.

This doesn't apply in an environment where you give users full access to install whatever extensions they choose. **But if the browser is the new endpoint, this is a bit like all your users being local admins — you're asking for trouble.** And locking down extensions in your organizations is something that can be achieved using native tools if you're, for example, a **Chrome Enterprise** customer. Audit your users once, approve only what's needed, and require further approval to install new extensions.

Identity is the prize, browser is the platform — and phishing is the weapon of choice

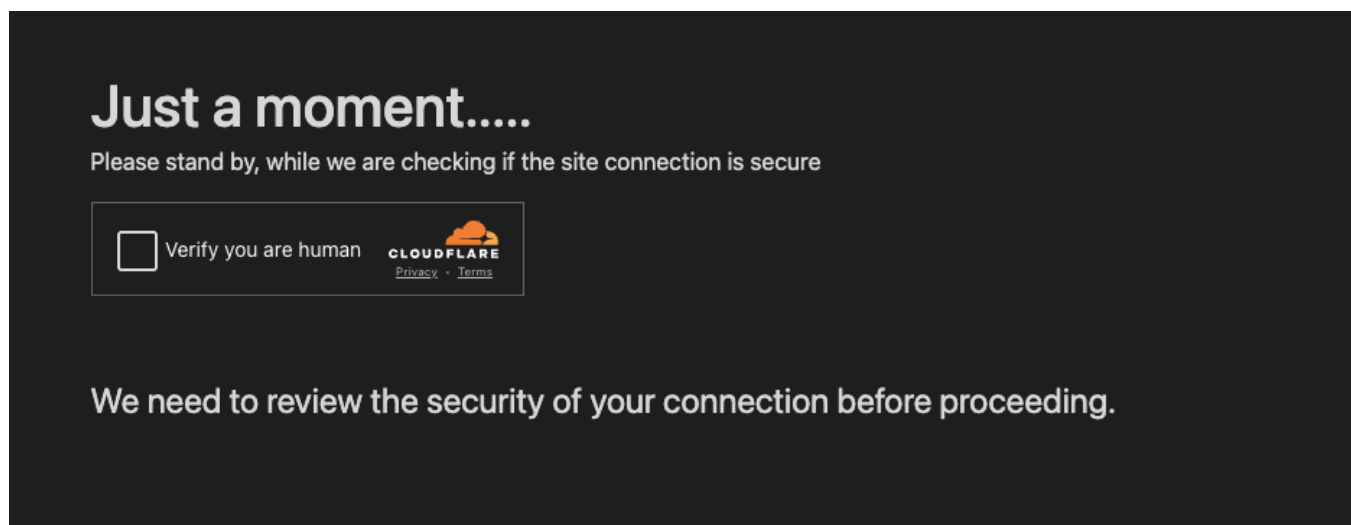
But the technique that's STILL driving the most impactful identity-driven breaches? **It's phishing.** Phishing for credentials, sessions, OAuth consent, authorization codes. Phishing via email, instant messenger, social media, malicious Google ads... it all happens in, or leads to, the browser.



—— All phishing roads lead to the browser, regardless of the delivery channel.

And modern phishing attacks are more effective than ever. Today, phishing operates on an industrial scale, using an array of obfuscation and detection evasion techniques to block email and network security tools from intercepting them. Probably the most common example today is the use of bot

protection (think CAPTCHA or Cloudflare Turnstile), using legitimate anti-spam features to block security tools.



—— Cloudflare Turnstile is a simple way for security teams to prevent automated analysis — it should probably come with a trigger warning for incident responders.

The latest generation of fully customized AitM phishing kits are dynamically obfuscating the code that loads the web page, implementing custom CAPTCHA, and using runtime anti-analysis features, making them increasingly difficult to detect. The ways in which links are delivered has also increased in sophistication, with more delivery channels (as we showed above) and the use of legitimate SaaS services for camouflage.

And the latest trends indicate that attackers are responding to increasingly hardened IdP/SSO configuration by exploiting alternative phishing techniques that **circumvent MFA and passkeys**, most commonly by **downgrading to a phishable backup authentication method** — which you can see in action below, and [read more about here](#).

Demonstrating an MFA downgrade attack using Evilginx



Identities are the lowest-hanging fruit for attackers to aim for

The goal of the modern attacker, and the easiest way into your business's digital environment, is to compromise identities. Whether you're dealing with phishing attacks, malicious browser extensions, or infostealer malware, the objective remains the same — account takeover.

Organizations are dealing with a vast and vulnerable attack surface consisting of:

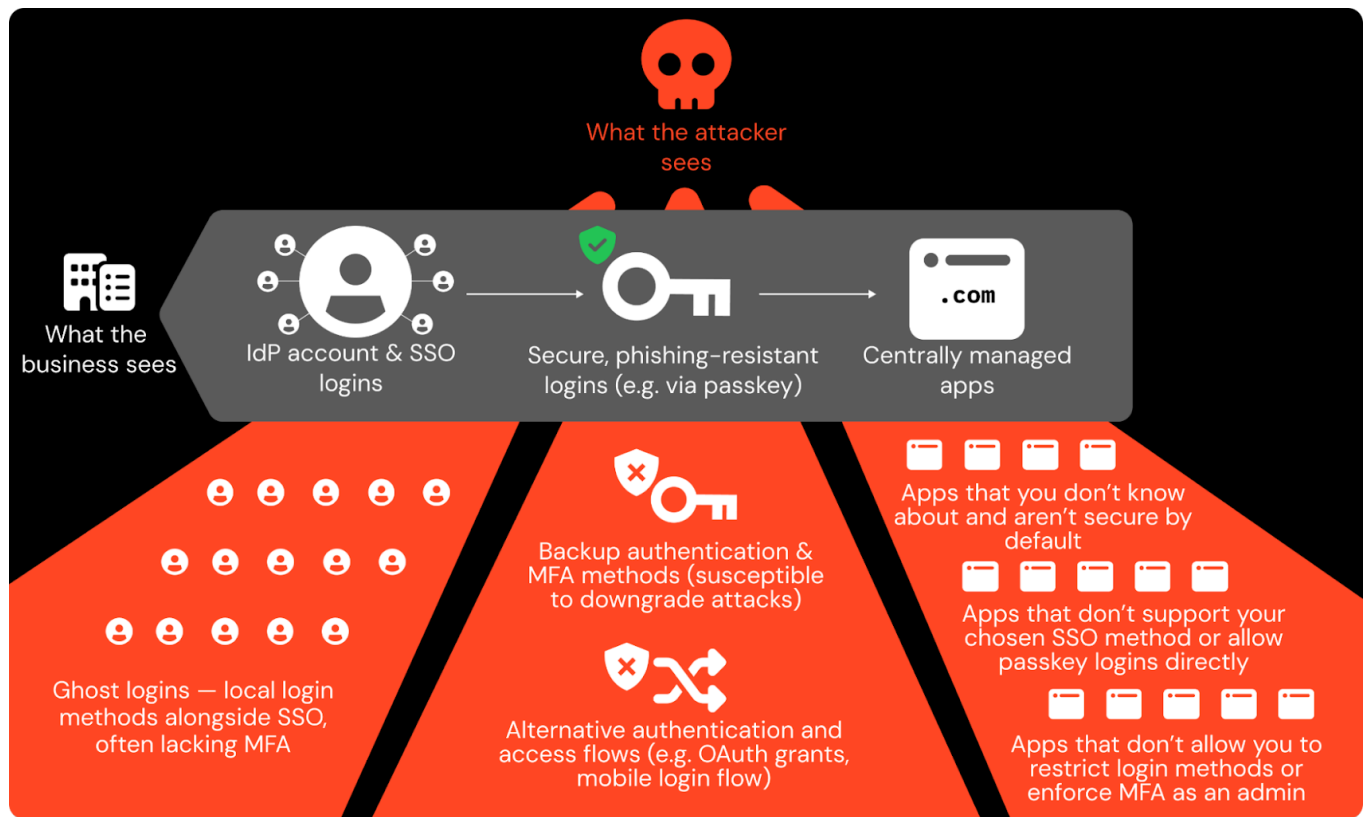
- **Hundreds of applications, with thousands of accounts** spread across the app estate.
- Accounts vulnerable to MFA-bypass phishing kits, because they are using a login method that is not phishing-resistant, or because **the login method can be downgraded**.
- Accounts with a weak, reused, or breached password and no MFA altogether (usually the result of a forgotten-about **ghost login**).
- Bypassing the authentication process entirely to evade otherwise phishing-resistant authentication methods, by abusing features like **API key creation, app-specific passwords, OAuth consent phishing, cross-IdP impersonation**, and more.



— A 1,000 user organization has over 15,000 accounts with various configurations and associated vulnerabilities.

A key driver of identity vulnerability is the **huge variance in the configurability of accounts per application**, with different levels of centralized visibility and security control of identities provided — for example, while one app can be locked down to only accept SSO logins via SAML and automatically remove any unused passwords, another provides no control or visibility of login method or MFA status (another big driver of the **Snowflake** breaches last year). Unfortunately, as a by-product of product-led growth and something that is compounded by every new SaaS startup that hits the market, this situation doesn't look like it's going to change anytime soon.

The end result is that identities are misconfigured, invisible to the security team, and routinely exploited by commodity attacker tooling. It's no surprise that they're the primary target for attackers today.



— Ghost logins, AitM phishing, downgrade attacks, and app-level configuration issues are fuelling identity-based breaches.

The solution: The browser as a telemetry source and control point

Because identity attacks play out in the browser, it's the perfect place for security teams to observe, intercept, and shut down these attacks.

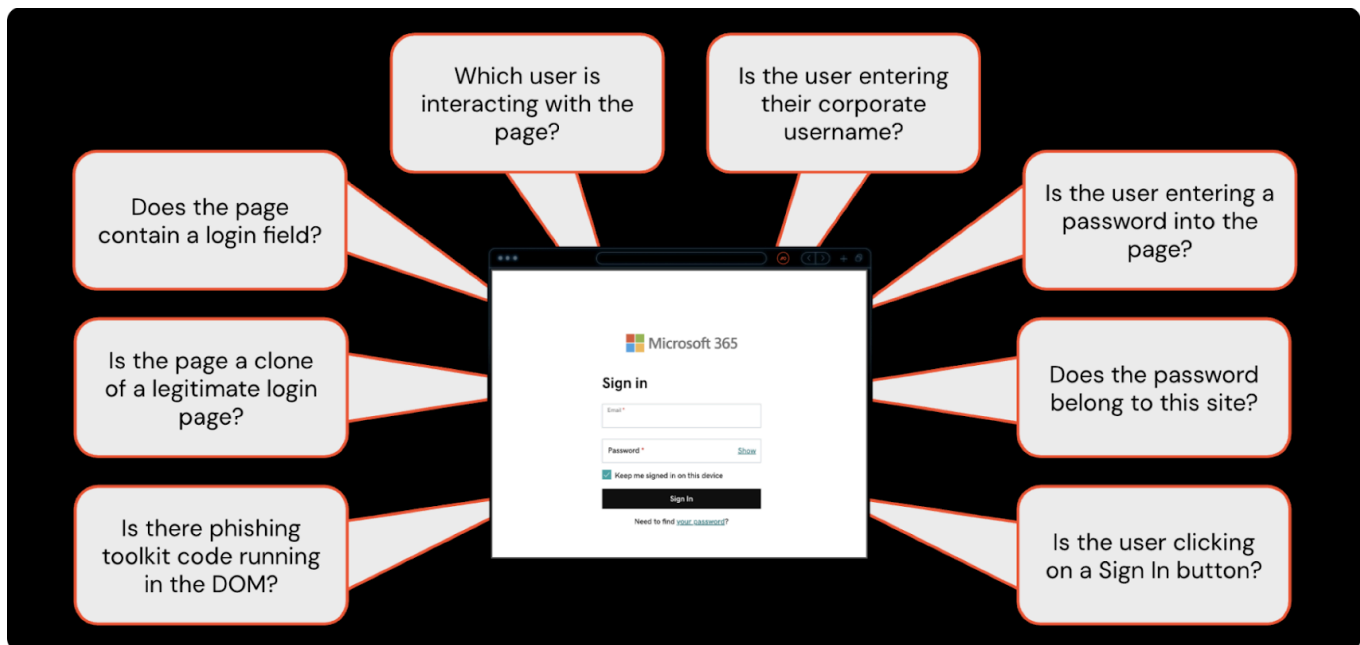
The browser has a number of advantages over the different places where identity can be observed and protected, because:

- You aren't limited to the apps and identities directly connected to your IdP (a fraction of your workforce identity sprawl).
- You aren't limited to the apps that you know about and manage centrally — you can observe every login that passes through the browser.
- You can observe all the properties of a login, including the login method, MFA method, etc. You'd otherwise need API access to *maybe* get this information (depending on whether an API is provided and whether this specific data can be interrogated, also not standard for many apps).

It's obvious with all that we've covered so far that fixing every identity vulnerability is an ominous task — the SaaS ecosystem itself is working against you. **This is why detecting and responding to identity attacks is essential.** Because identity compromise almost always involves phishing or social engineering a user to perform an action in their browser (with some exceptions — like the [Scattered Spider-related help desk attacks](#) seen recently), it's also the perfect place to monitor for and intercept attacks.

In the browser, you gather deep, contextualized information about page behavior and user inputs that can be used to detect and shut down risky scenarios in real time. Take the example of phishing pages. Because Push operates in the browser, it sees everything:

- The page layout
- Where the user came from
- The password they enter ([as a salted, abbreviated hash](#))
- What scripts are running
- And where credentials are being sent



— Being in the browser gives you unrivalled visibility of phishing page activity and user behavior.

Conclusion

Identity attacks are the biggest unsolved problem facing security teams today and the leading cause of security breaches. At the same time, the browser presents security teams with all the tools they need to prevent, detect, and respond to identity-based attacks — proactively by finding and fixing identity vulnerabilities, and reactively by detecting and blocking attacks against users in real time.

Organizations need to move past the old ways of doing identity security — relying on MFA attestations, identity management dashboards, and [legacy email and network anti-phishing tools](#). And there's no better place to stop these attacks than in the browser.

Find out more

Push Security's browser-based security platform provides comprehensive detection and response capabilities against the leading cause of breaches. Push blocks identity attacks like AiTM phishing, credential stuffing, password spraying and session hijacking using stolen session tokens. You can also use Push to find and fix identity vulnerabilities across the apps that your employees use, like ghost logins, SSO coverage gaps, MFA gaps, vulnerable passwords, risky OAuth integrations, and more.

If you want to learn more about how Push helps you to detect and stop attacks in the browser, [book some time with one of our team for a live demo](#).

Found this article interesting? This article is a contributed piece from one of our valued partners. Follow us on [Google News](#), [Twitter](#) and [LinkedIn](#) to read more exclusive content we post.

CYBERSECURITY WEBINARS

[Hidden Attack Paths You're Missing](#)

Learn to Find Hidden Vulnerabilities in Autonomous AI Agents

A practical deep dive into securing AI agents against real-world attack paths beyond the model itself.

[Join the Session](#)